Telco Customer Churn Analysis

Written Report | Feature Engineering, Scaling, Model Comparison and Algorithm Justification | Week 3 Assignment, Data Science Internship Program

1. Data Preparation

The raw dataset contains 7043 customer records and 21 columns. Inspection with df.info() showed that TotalCharges was stored as an object column rather than a numeric one, because 11 rows held blank string values instead of proper missing values. Every one of those 11 blank rows had a tenure of zero, meaning the customer had not yet completed a full billing cycle when the record was captured. TotalCharges was therefore converted to numeric using pandas.to_numeric with errors set to coerce, and the resulting nulls were filled with zero rather than an average or median value, since zero genuinely reflects a brand new customer who has not been billed yet, rather than a value that should be estimated.

The customerID column was dropped before modeling because it is a unique identifier with no predictive signal, and keeping it risks acting as a data leak if it were accidentally encoded. The target variable, Churn, was confirmed to contain only the two expected labels Yes and No, then mapped to 1 and 0. The class split is 5174 customers who did not churn versus 1869 who did, roughly 73 percent against 27 percent, which is a mild but real imbalance. This is the reason accuracy alone is not treated as sufficient in this project; precision, recall, F1 score and ROC AUC were tracked for every model.

2. Feature Engineering Decisions

Three new features were engineered beyond the raw columns provided in the dataset, each chosen to expose a pattern that a plain linear combination of raw columns would not easily capture.

tenure_group. Raw tenure in months was bucketed into New (0 to 12 months), Established (13 to 48 months) and Long term (49 months and above). Churn risk drops sharply and nonlinearly over a customer's first year and then flattens out, a pattern that a single coefficient on raw tenure cannot represent well. The bar chart of churn rate by tenure group confirmed the intuition: churn rate fell from roughly 47 percent for new customers to about 10 percent for long term customers.

total_services. This feature counts how many of the nine optional add on and connectivity services a customer subscribes to, out of PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV and StreamingMovies. Customers who subscribe to more services tend to be more invested in the provider, so this count was expected to correlate negatively with churn. The plot of churn rate against total_services supported this, showing generally lower churn among customers with more services subscribed.

monthly_to_tenure_ratio. This is MonthlyCharges divided by tenure plus one. It approximates how concentrated a customer's spending is relative to how long they have been with the company, which can flag customers whose bill is high relative to their history, a pattern often associated with price sensitive churn. Churned customers clustered at higher values of this ratio in the scatter plot produced in the notebook.

Categorical encoding followed a simple rule. Contract has a genuine order (Month to month is shorter and less committed than One year, which is shorter than Two year), so it was label encoded as 0, 1 and 2. Every other categorical column, including gender, InternetService, PaymentMethod and the new tenure_group feature, is nominal with no natural ranking, so those columns were one hot encoded with the first category dropped to avoid redundancy. After encoding, the feature matrix contained 33 predictor columns.

3. Feature Scaling Justification

Tree based models such as Decision Tree, Random Forest, Gradient Boosting and XGBoost split on thresholds one feature at a time, so the absolute scale of a feature does not affect them. Distance based and margin based models behave very differently: K Nearest Neighbors measures raw distance between points, and Support Vector Machine

optimizes a margin that is sensitive to the relative scale of every dimension. Logistic Regression also converges faster and produces more stable coefficients on scaled inputs. The continuous columns that genuinely needed scaling were tenure, MonthlyCharges, TotalCharges, total_services and monthly_to_tenure_ratio. The one hot encoded columns are already 0 or 1 and were left untouched.

Both StandardScaler and MinMaxScaler were fit, and in each case the fit was performed only on the training split, then applied to transform both the training and test splits. Fitting a scaler on the full dataset, or on the test set, would let statistics from the held out data leak into training, producing an optimistic and unrealistic estimate of how the model performs on genuinely unseen data, which defeats the purpose of a test set. StandardScaler was carried forward into the final modeling stage. TotalCharges and MonthlyCharges are right skewed with a meaningful number of higher value outliers, and StandardScaler centers each feature around a mean of zero with unit variance without artificially compressing every value into a fixed zero to one range the way MinMaxScaler does. MinMaxScaler was kept as a documented alternative but was not required by any of the models trained here, since none of them strictly need bounded, nonnegative inputs.

The train test split used an 80 to 20 ratio with stratification on Churn, so the training set and the test set both preserve the same roughly 26.5 percent churn rate seen in the full dataset.

4. Model Comparison

Eight classifiers spanning five model families were trained: Logistic Regression as the linear baseline, Decision Tree and Random Forest as tree based models, Gradient Boosting and XGBoost as boosted tree models, K Nearest Neighbors as the distance based model, Support Vector Machine as the margin based model, and Gaussian Naive Bayes as the probabilistic model. Tree based and Naive Bayes models used the unscaled features, while Logistic Regression, K Nearest Neighbors and Support Vector Machine used the standardized features. Every model was evaluated on the held out test set using accuracy, precision, recall, F1 score and ROC AUC, and cross checked with five fold stratified cross validated ROC AUC.

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC	CV ROC AUC
Logistic Regression	0.8034	0.6644	0.5241	0.5859	0.8461	0.8485
Gradient Boosting	0.7977	0.6551	0.5027	0.5688	0.8437	0.8479
K Nearest Neighbors	0.7878	0.6119	0.5481	0.5783	0.8277	0.8293
Random Forest	0.7885	0.6310	0.4893	0.5512	0.8254	0.8277
Naive Bayes	0.6899	0.4623	0.8369	0.5957	0.8200	n/a
XGBoost	0.7715	0.5697	0.5321	0.5503	0.8080	n/a
Support Vector Machine	0.8005	0.6784	0.4813	0.5631	0.7950	n/a
Decision Tree	0.7395	0.5107	0.5027	0.5067	0.6690	n/a

Results are sorted by test set ROC AUC. Logistic Regression finished first on both the held out test set and on five fold cross validation, narrowly ahead of Gradient Boosting. K Nearest Neighbors and Random Forest followed closely behind, and Decision Tree was clearly the weakest single model, consistent with a single unconstrained tree overfitting the training data.

5. Algorithm Research and Justification

5.1 What each family assumes

Logistic Regression models the log odds of churn as a linear combination of the input features. It assumes the classes are roughly linearly separable and is sensitive to feature scale because its coefficients are found through

gradient based optimization. Decision Tree makes no assumption about linearity or scale and can capture interactions automatically, but a single tree tends to overfit and is unstable. Random Forest trains many trees on bootstrapped samples and averages their votes, reducing variance while keeping the same insensitivity to scale. Gradient Boosting and XGBoost build trees sequentially, each one correcting the errors of the previous ensemble, and generally achieve strong raw performance on structured tabular data at the cost of more hyperparameters and a higher risk of overfitting if left unconstrained. K Nearest Neighbors assumes that proximity in feature space implies similarity in outcome and is highly sensitive to scale. Support Vector Machine finds the boundary that maximizes the margin between classes and also depends on scaled inputs. Gaussian Naive Bayes assumes features are conditionally independent given the class, an assumption clearly violated here since a customer who lacks internet service automatically lacks every internet dependent add on.

5.2 What the theory predicted versus what happened

General guidance on tabular classification favors boosted tree ensembles over linear models, since boosting can represent nonlinear thresholds and feature interactions without being told about them explicitly. That was the expectation going into Task 4. The measured results tell a narrower story: Logistic Regression beat Gradient Boosting by less than half a percentage point of ROC AUC on both the test set and cross validation, a small but consistent margin rather than random noise.

Three properties of this specific pipeline plausibly explain the reversal. First, the Task 2 feature engineering already absorbed much of the nonlinearity that boosting is normally needed for. tenure_group turns a sharply nonlinear tenure relationship into a small set of nearly linear steps, and total_services and monthly_to_tenure_ratio compress multi column interactions into single features a linear model can weight directly, which narrows the usual gap between linear and boosted models. Second, roughly seven thousand rows and thirty three engineered columns is a modest size for a boosting algorithm; ensembles typically pull further ahead as row count and interaction complexity grow, so with this little data a lower variance linear model can hold its own. Third, Gradient Boosting was trained here with scikit learn's default hyperparameters rather than a tuned learning rate, tree depth or number of estimators, so its true ceiling was likely not fully reached in this run.

5.3 Chosen algorithm, limitation, and an alternative scenario

Based on the measured results, Logistic Regression is the empirically justified choice for this specific, already well engineered dataset of this size. Its main limitation is that its performance leans heavily on the quality of the manual feature engineering behind it; the model can only combine features linearly, so tenure_group, total_services and monthly_to_tenure_ratio were doing real work that the model itself cannot discover on its own. If those engineered columns were removed, or if this pipeline were applied to a larger, less manually curated dataset with many more raw columns and interactions, that advantage would likely disappear.

That is the scenario where Gradient Boosting or XGBoost would become preferable instead. With more training rows, a wider and less curated feature set, or proper tuning of learning rate, tree depth and number of estimators, boosting's capacity to learn interactions and nonlinear splits on its own would likely close the small gap observed here and move ahead of the linear model, consistent with both the scikit learn documentation on ensemble methods, which describes gradient boosted trees as generally delivering strong predictive accuracy on structured data (Pedregosa et al., scikit learn documentation, Ensemble methods section), and the original XGBoost paper by Chen and Guestrin, which demonstrates strong benchmark results for regularized gradient boosting while noting that its advantage depends on adequate data volume and tuning (Chen, T. and Guestrin, C., XGBoost: A Scalable Tree Boosting System, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016).

6. Conclusion

This project moved from a messy, mixed type CSV to eight properly evaluated classifiers, guided at each step by an explicit justification. TotalCharges nulls were filled based on the business meaning of a zero tenure customer, categorical variables were encoded according to whether a genuine order existed, StandardScaler was chosen for its robustness to skew, and the final algorithm recommendation weighed the measured metrics against the theoretical properties of each model family. Logistic Regression narrowly outperformed Gradient Boosting on this run, despite

the literature generally favoring boosting on tabular data, and the most defensible explanation combines the strength of the Task 2 feature engineering, the modest size of the dataset, and untuned boosting hyperparameters. With more data or proper tuning, Gradient Boosting remains the stronger long term choice, while Logistic Regression stays attractive in settings that also value transparent, easily explained decisions.